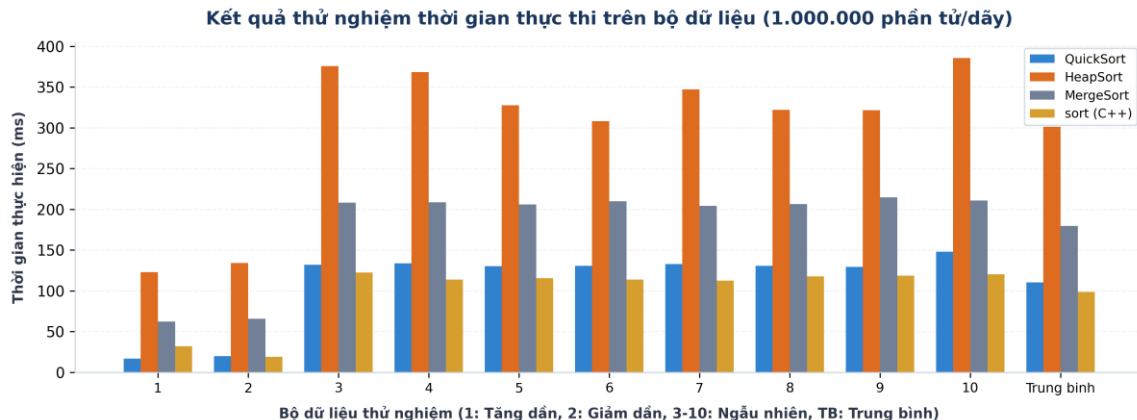


BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Trần Tiến Dũng - 25520381**Nội dung báo cáo:** So sánh các thuật toán sắp xếp trên nhiều bộ dữ liệu khác nhau**I. Kết quả thử nghiệm****1. Bảng thời gian thực hiện**

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	16.79	122.70	62.39	31.92
2	19.75	134.13	65.72	18.87
3	131.74	375.38	207.86	122.26
4	133.47	368.16	208.62	113.60
5	130.15	327.64	205.88	115.23
6	130.74	308.04	209.97	113.90
7	132.63	347.23	204.29	112.50
8	130.62	321.75	206.27	117.74
9	129.35	321.45	214.56	118.27
10	147.85	385.53	210.57	120.31
Trung bình	110.31	301.20	179.61	98.46

2. Biểu đồ (cột) thời gian thực hiện**II. Kết luận và Nhận xét kết quả thực nghiệm:**

- HeapSort chậm nhất trong 4 thuật toán (trung bình 301.20 ms): Mặc dù sắp xếp tại chỗ $O(1)$ bộ nhớ phụ, nhưng thao tác heapify truy cập bộ nhớ không tuần tự (nhảy cách quãng $2i+1$, $2i+2$) gây hiện tượng trượt bộ nhớ đệm (cache miss) liên tục trên mảng 1 triệu phần tử.
- MergeSort có tính ổn định cao nhưng chịu chi phí sao chép bộ nhớ (trung bình 179.61 ms): Thời gian ổn định trong mọi trường hợp, nhưng việc dùng mảng phụ $O(N)$ khiến tốc độ chậm hơn QuickSort và `std::sort`.
- QuickSort (chọn chốt ở giữa, phân hoạch Hoare) có tốc độ rất ấn tượng (trung bình 110.31 ms): Đặc biệt với mảng tăng dần (16.79 ms) và giảm dần (19.75 ms), việc chọn chốt giữa chia đôi mảng hoàn hảo và tận dụng tối đa tính cục bộ bộ nhớ đệm (cache locality) của CPU.

- `std::sort` (C++ STL - Introsort) đạt tốc độ nhanh nhất (trung bình 98.46 ms): Thuật toán lai ghép thông minh giữa QuickSort, HeapSort (khi đệ quy sâu $> 2\log N$) và InsertionSort (khi mảng ≤ 16), kết hợp tối ưu hoá phân cứng tuyệt vời.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
 2. Mã nguồn
 3. Dữ liệu thử nghiệm
- **Link github:** <https://github.com/ttdapril25-dev/IT003---Week-4.git>